

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

TECNOLOGÍAS DE LA INFORMACIÓN DESARROLLO DE SOFTWARE

EJEMPLO INTEGRADOR S9

ARQUITECTURA DE AUTENTICACIÓN Y CONTROL DE ROLES

PRESENTA:

OSCAR ESPINOZA LANDETA

8 “A”

A CARGO:

ING. JOSÉ MIGUEL CARRERA PACHECO



REPORTE TÉCNICO DE ARQUITECTURA DE SOFTWARE

Diseño de Seguridad: Autenticación y Control de Roles en Aplicaciones Web Modernas

Resumen Ejecutivo

El presente reporte define los lineamientos arquitectónicos y de ciberseguridad para la implementación de un sistema robusto de autenticación (AuthN) y autorización (AuthZ) en las futuras plataformas web de la organización. Se evalúan tecnologías estándar (JWT, Sesiones, OAuth 2.0), se analizan los vectores de ataque más comunes y se propone una arquitectura híbrida basada en tokens (Access + Refresh) junto con un modelo de Control de Acceso Basado en Roles (RBAC).

1. Conceptos Fundamentales

Para garantizar un diseño seguro, es imperativo establecer la frontera arquitectónica entre la validación de identidad y la asignación de privilegios.

- **Autenticación (AuthN):** Es el mecanismo mediante el cual el sistema verifica la identidad de una entidad (usuario, dispositivo, microservicio). Responde a la pregunta: "*¿Eres quien dices ser?*".
- **Autorización (AuthZ):** Es el proceso posterior a la autenticación que determina si la identidad verificada posee los privilegios necesarios para interactuar con un recurso específico. Responde a la pregunta: "*¿Tienes permiso para realizar esta acción?*".
- **Diferencia Clave:** La autenticación es el control de acceso a la plataforma (la llave del edificio); la autorización es el control de acceso a las funcionalidades (la llave de una oficina específica dentro del edificio).

2. Tecnologías de Autenticación

2.1. JWT (JSON Web Tokens)

- **Funcionamiento:** Estándar (RFC 7519) que transmite identidad y privilegios en un objeto JSON firmado criptográficamente (header, payload, signature).
- **Ventajas:** Arquitectura sin estado (*stateless*). El servidor no consulta la base de datos para validar la sesión, optimizando la escalabilidad y reduciendo la latencia.
- **Desventajas:** Dificultad intrínseca para revocar un token antes de su expiración. El tamaño del token puede inflar las cabeceras HTTP si el payload crece.
- **Casos de uso:** Arquitecturas de microservicios, Single Page Applications (SPAs) y APIs

RESTful de alto tráfico.

2.2. Sesiones (Stateful Authentication)

- **Funcionamiento:** El servidor crea un registro en memoria/DB y devuelve un identificador único (Session ID) almacenado en una cookie del cliente.
- **Ventajas:** Control absoluto e inmediato. Revocar el acceso es tan simple como borrar el registro en el backend. Los datos del usuario permanecen ocultos al cliente.
- **Desventajas:** Penaliza la escalabilidad. Requiere sincronización del estado en infraestructuras distribuidas (ej. usando clústeres de Redis).
- **Casos de uso:** Sistemas monolíticos heredados, plataformas bancarias de alta criticidad, aplicaciones donde la invalidación instantánea es mandatoria.

2.3. OAuth 2.0

- **Funcionamiento:** Marco de trabajo de delegación de acceso. Permite a una aplicación interactuar con recursos de un usuario en otro servicio mediante el intercambio de Access Tokens, sin exponer credenciales.
- **Ventajas:** Mejora la experiencia de usuario (Single Sign-On). Evita el almacenamiento descentralizado de contraseñas.
- **Desventajas:** Complejidad alta en la implementación de sus flujos (Authorization Code, Client Credentials). Vulnerable a configuraciones defectuosas (Open Redirects).
- **Casos de uso:** Integraciones B2B, autenticación social ("Login with Google/Microsoft"), ecosistemas corporativos centralizados (IAM).

3. Comparación de Estrategias y Criterios de Selección

Característica	JWT (Stateless)	Sesiones (Stateful)	OAuth 2.0 / OIDC
Almacenamiento de Estado	Cliente (Token)	Servidor (Memoria/Redis/DB)	Servidor de Autorización
Escalabilidad	Muy Alta	Media	Alta
Revocación Inmediata	Compleja (Requiere Blacklists)	Simple y directa	Depende del Proveedor
Complejidad Arquitectónica	Media	Baja	Alta

Directrices Arquitectónicas:

- **¿Cuándo usar tokens?** En ecosistemas distribuidos (APIs sirviendo a Web, iOS, Android simultáneamente) donde las consultas a base de datos por cada petición HTTP representan un cuello de botella inaceptable.
- **¿Cuándo usar sesiones?** En monolitos altamente acoplados con base de usuarios

controlada donde el costo de infraestructura para gestionar sesiones es menor que el riesgo de no poder revocar accesos instantáneamente.

- **Conclusión para el Proyecto Actual:** Se recomienda una **arquitectura híbrida** (detallada en la Sección 6 y 7).

4. Riesgos de Seguridad y Estrategias de Mitigación

Un diseño deficiente expone el sistema a vulnerabilidades críticas. A continuación, los riesgos y la arquitectura preventiva:

1. Robo de Sesión (vía XSS):

- *Riesgo:* Ejecución de JavaScript malicioso en el navegador del cliente para extraer tokens almacenados en localStorage.
- *Mitigación:* Almacenar identificadores críticos (Refresh Tokens o Session IDs) estrictamente en cookies con los atributos HttpOnly, Secure y SameSite=Strict.

2. Ataques de Fuerza Bruta:

- *Riesgo:* Intentos masivos para adivinar credenciales en el endpoint de *login*.
- *Mitigación:* Implementar *Rate Limiting* por IP en la capa del API Gateway, políticas de bloqueo temporal de cuentas (Account Lockout) y obligatoriedad de MFA (Multi-Factor Authentication) en roles administrativos.

3. Almacenamiento Inseguro de Tokens/Credenciales:

- *Riesgo:* Fuga de bases de datos que compromete contraseñas en texto plano o el uso de claves secretas débiles (HS256) para firmar JWTs.
- *Mitigación:* Derivación de claves usando Argon2 o bcrypt. Uso de criptografía asimétrica (RS256) para la firma de JWTs en entornos de microservicios.

4. Mala Gestión de Permisos (IDOR):

- *Riesgo:* Insecure Direct Object Reference. Un usuario cambia el ID en la URL (/api/docs/1 a /api/docs/2) y accede a información privada.
- *Mitigación:* Verificación estricta en la capa de persistencia cruzando el ID extraído del token criptográfico contra el propietario real del recurso en base de datos.

5. Escalación de Privilegios:

- *Riesgo:* Inyección de parámetros (Mass Assignment) durante la actualización de perfil para modificar el atributo role_id.
- *Mitigación:* Aplicación estricta de DTOs (Data Transfer Objects) y sanitización de *payloads*. El endpoint de actualización de usuario estándar debe ignorar o rechazar cualquier intento de mutación de roles.

5. Diseño de Roles y Permisos (RBAC)

El Control de Acceso Basado en Roles (Role-Based Access Control) se implementará mediante una estructura normalizada de base de datos que desacopla la identidad de las reglas de negocio.

● Entidades Core:

- Users: Identidad digital (Autenticación).
- Roles: Agrupación semántica de funciones (ej. Auditor).

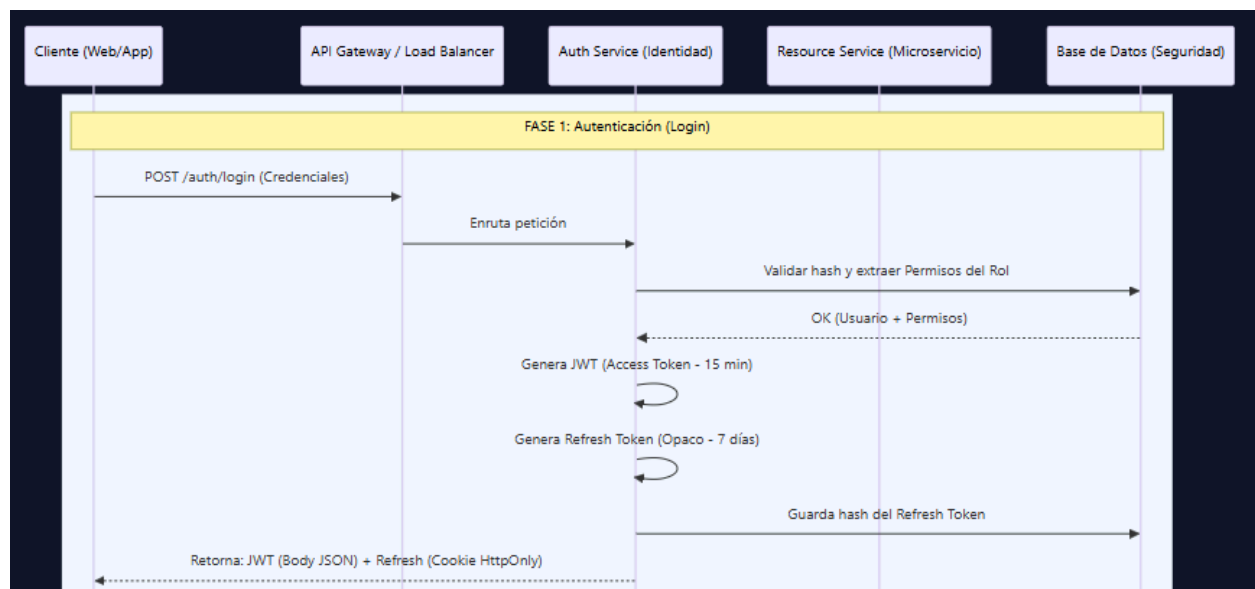
- Permissions: Reglas granulares de ejecución (ej. export:reports).
- **Modelo Conceptual:** El sistema valida permisos, no roles. Esto asegura que si un rol cambia de responsabilidades, el código fuente no necesita modificaciones, solo la base de datos.

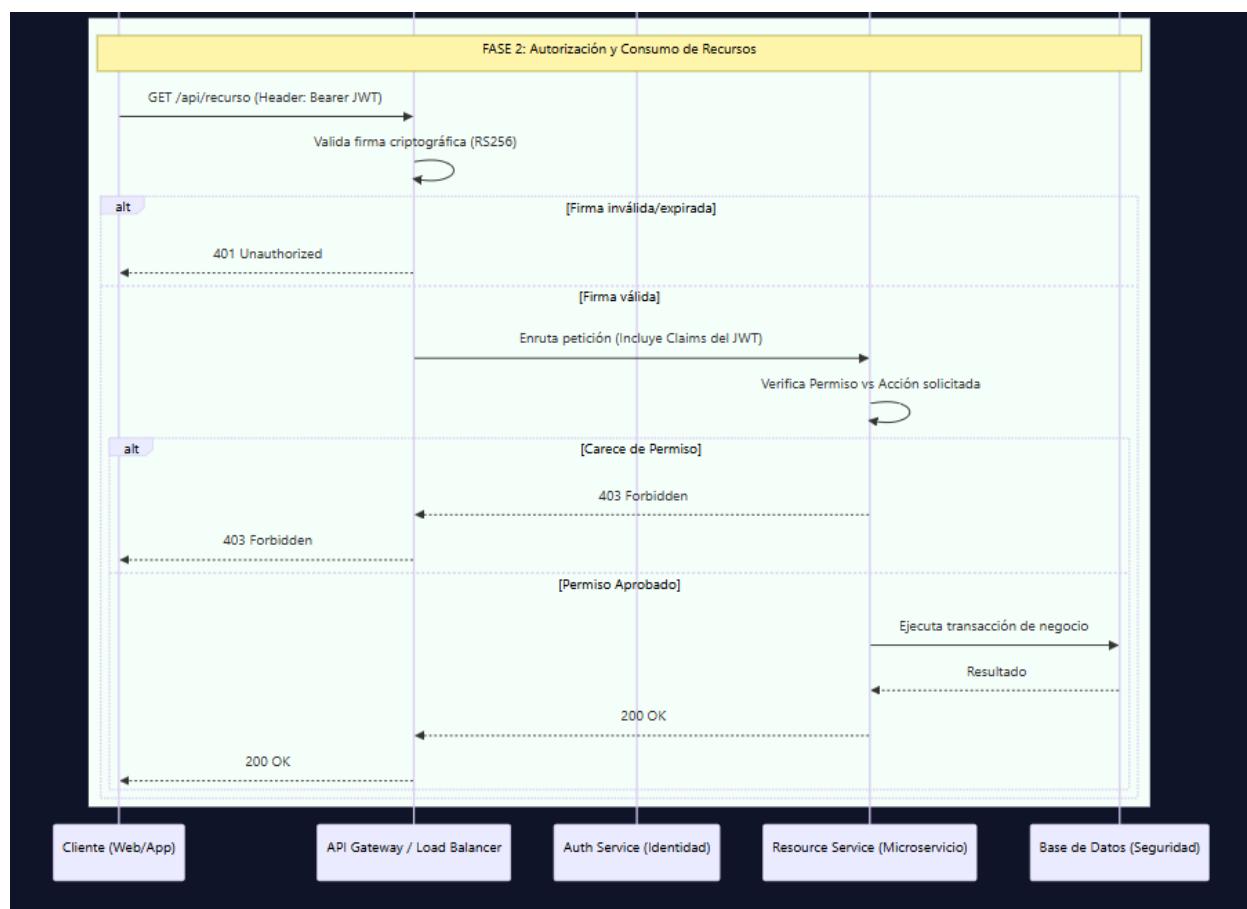
Matriz de Ejemplo:

- **Rol: Administrador**
 - Permisos: users:create, users:delete, system:config, data:export.
- **Rol: Usuario (Estándar)**
 - Permisos: profile:read, profile:update, posts:create.
- **Rol: Invitado**
 - Permisos: posts:read.

6. Diseño de Arquitectura de Autenticación

El siguiente diagrama ilustra el flujo híbrido recomendado, balanceando seguridad (revocación) y rendimiento (stateless).





7. Justificación de Decisiones Técnicas

La arquitectura definida en la sección 6 adopta un **Enfoque Híbrido (JWT + Opaque Refresh Token rotatorio)**.

1. **Por qué es adecuado:** Los microservicios de recursos (Resource Service) no necesitan conectarse a la base de datos para saber quién es el usuario ni qué puede hacer, ya que confían en la firma del API Gateway sobre el JWT. Esto permite escalar los microservicios independientemente y absorber picos masivos de tráfico.
2. **Mitigación de riesgos integrada:** * El JWT expira en 15 minutos. Si es robado, la ventana de ataque es mínima.
 - El Refresh Token vive en una cookie HttpOnly (inmune a XSS) y su estado real reside en la DB.
 - En caso de compromiso de cuenta, el Administrador invalida el Refresh Token en la base de datos. A los 15 minutos máximos, el JWT caduca y el atacante no podrá renovarlo, perdiendo acceso definitivo.

8. Plan de Implementación (Entregables y Tareas)

Para iniciar el ciclo de desarrollo ágil, se definen las siguientes épicas e *issues* iniciales para el

equipo de ingeniería:

- [] **SEC-01 (Infraestructura):** Configurar API Gateway para centralizar la validación de firmas JWT usando algoritmos asimétricos (RS256).
- [] **SEC-02 (Base de Datos):** Diseñar e implementar esquemas relacionales para RBAC (users, roles, permissions, tablas puente).
- [] **SEC-03 (Backend):** Implementar servicio de autenticación con rotación de Refresh Tokens y entrega segura mediante cabeceras Set-Cookie.
- [] **SEC-04 (Seguridad Perimetral):** Configurar reglas de *Rate Limiting* en endpoints públicos (/login, /register, /forgot-password).
- [] **SEC-05 (Backend/Middleware):** Desarrollar interceptores/middlewares a nivel de controladores que extraigan permisos del JWT y apliquen políticas de bloqueo (HTTP 403).
- [] **SEC-06 (QA/Security):** Ejecutar pruebas de penetración automatizadas verificando resistencia contra IDOR y escalación de privilegios en endpoints de escritura.